FinCore – Banking System and Transaction Management Database – Design and Implementation of a MySQL-based Relational Database for Customers, Accounts, Branches and Loans using SQL, Views, Stored Procedures, Triggers and Functions

A PROJECT REPORT

Submitted by

Bhumi Jindal (26MCD10071)

Submitted to

Mrs. Winky Bhatia

in partial fulfillment for the award of the degree of

Master of Computer Applications

IN

Data Science

2026-28

Chandigarh University
2026-28

BONAFIDE CERTIFICATE

Certified that this project report **"FinCore – Banking System and Transaction Management Database"** is the bonafide work of **"Bhumi Jindal (26MCD10071)"** who carried out the project work under my/our supervision. The content and findings presented herein are authentic and reflect the original work conducted during the course of this project.

SIGNATURE	**SIGNATURE**
Name: Bhumi Jindal	Name: Mrs. Winky Bhatia
UID: 26MCD10071	Designation: Asst. Professor
Department: MCD - 2/A	SUBJECT CODE: E20679

Submitted for the project viva-voce examination held on **____ / ____ / 2026**

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to all the individuals who have contributed to the success of this project. Designing a relational database for a bank – with its keys, relationships, business rules and audit requirements – presented several challenges, and I am immensely grateful for the support and guidance I received along the way.

I extend my sincere appreciation to my project supervisor, Mrs. Winky Bhatia, for her invaluable support and encouragement throughout the project. I also owe a special debt of gratitude to her, whose guidance, expertise and motivation were instrumental in overcoming obstacles and achieving the goals of this work.

I would also like to thank all the faculty members of the University Institute of Computing for their valuable suggestions and feedback, which helped me improve the database design and explore new ideas.

A heartfelt thank you goes to my parents for their unwavering support and encouragement. I am grateful to all those who, directly or indirectly, contributed to this project.

Bhumi Jindal
UID: 26MCD10071

TABLE OF CONTENTS

Abstract ..... **iv**
CHAPTER 1: INTRODUCTION **1**
1.1. Project Scope and Project Planning 1
1.2. Client Identification and Recognition of need 1
1.3. Recognition and Knowledge of relevant contemporary issues 1
1.4. Task Identification 1
1.5. Technologies Used 1
1.6. Timeline of the project 1
CHAPTER 2: LITERATURE REVIEW/BACKGROUND STUDY **2**
2.1 Timeline of the reported problem 2
2.2 Existing solutions 2
2.3 Bibliometric analysis 2
2.4 Review Summary 2
2.5 Problem Definition 2
2.6 Goals/ Objectives 2
CHAPTER 3: DESIGN FLOW/PROCESS **3**
3.1 Evaluation & Selection of Features 3
3.2 Design Constraints 3
3.3 Analysis of Features and finalization subject to constraints 3
3.4 Design Flow 3
3.5 Design Selection 4
3.6 Implementation plan/methodology 4
CHAPTER 4: RESULTS ANALYSIS AND VALIDATION **5**
4.1 Implementation of the solution 5
4.2 Summary of findings and validation 8
CHAPTER 5: CONCLUSION AND FUTURE WORK **9**
5.1 Conclusion 9
5.2 Future Scope 9
APPLICATION CODE OF PROJECT **10**
REFERENCES **11**
APPENDIX **11**
USER MANUAL **11**
GITHUB REPOSITORY **11**

List of Figures

Figure 1: Development lifecycle of the project 1
Figure 2: Workflow and architecture of the database project 3
Figure 3: Entity-relationship (ER) diagram of the FinCore database 3
Figure 4: Tables created in bank_management_db (SHOW TABLES, DESCRIBE) 5
Figure 5: Sample data inserted into Account and Loan 5
Figure 6: SELECT with AND condition and multi-table JOIN 6
Figure 7: Loan JOIN and GROUP BY with HAVING 6
Figure 8: UPDATE and DELETE operations with verification 6
Figure 9: Output of the two views 7
Figure 10: Stored procedure sp_TransferFunds 7
Figure 11: Trigger audit log and overdraft protection 8
Figure 12: Output of the two user-defined functions 8
Figure 13: GitHub repository of the project 11

ABSTRACT

Banks manage millions of customers, accounts and loans, and every deposit, withdrawal and transfer must be recorded correctly, consistently and traceably. A well-designed relational database is the foundation of such a system. This project, **FinCore – Banking System and Transaction Management Database**, designs and implements a MySQL database named *bank_management_db* that stores banks, branches, customers, accounts and loans, and supports day-to-day banking operations through SQL.

The database consists of **10 normalised tables** plus one audit table: Bank, Branch, Customer, CustomerPhoneNumber, AccountType, Account, CustomerAccount, LoanType, Loan and CustomerLoan, with primary keys, foreign keys, UNIQUE and NOT NULL constraints, default values and three indexes. Many-to-many relationships (customers–accounts and customers–loans) are resolved with junction tables, the multivalued phone number is stored in its own table, and lookup tables hold account and loan types.

All four CRUD operations are demonstrated with 10 INSERT, 5 SELECT (WHERE, JOIN, GROUP BY, HAVING), 4 UPDATE and 2 DELETE statements. Advanced objects add business logic inside the database: **2 views** (customer account summary and branch performance), **2 stored procedures** (fund transfer and add customer), **2 triggers** (balance audit log and overdraft prevention) and **2 functions** (monthly interest and customer total balance). With the sample data, the bank holds total balances of **59,200.00** across five accounts and **585,000.00** of loans across three loans, and every balance change is captured in the audit table.

The script and output screenshots are published on GitHub. This report describes the scope, background, design, implementation, results and future scope of the project.

Keywords: MySQL, Relational Database, Banking System, SQL, Stored Procedure, Trigger, View, Function, Transaction Management.

List of Abbreviations

Abbreviation	Definition
SQL	Structured Query Language
DBMS / RDBMS	Database / Relational Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
CRUD	Create, Read, Update, Delete
PK / FK	Primary Key / Foreign Key
ER	Entity-Relationship
M:N	Many-to-Many relationship
ACID	Atomicity, Consistency, Isolation, Durability
KYC / AML	Know Your Customer / Anti-Money Laundering
UI	User Interface

CHAPTER 1.
INTRODUCTION

A bank is, at its core, an information system. Every customer, account, branch and loan is a record, and every deposit, withdrawal or transfer changes those records. If data is duplicated, inconsistent or can change without a trace, the bank loses money and trust. A relational database management system (RDBMS) solves this by storing data in related tables, enforcing rules through keys and constraints, and answering precise questions through SQL.

FinCore – Banking System and Transaction Management Database is a MySQL-based banking database for managing customers, accounts, branches, loans and transaction-related operations. It is built entirely in SQL: the schema (DDL), sample data and CRUD operations (DML), and advanced objects – views, stored procedures, triggers and functions – that keep rules such as *no overdraft* and *audit every balance change* inside the database itself.

1.1. Project Scope and Project Planning:

The scope covers the design and implementation of the *bank_management_db* database: 10 tables with constraints and three indexes; sample banks, branches, customers, accounts and loans; CRUD operations with joins, grouping and filtering; and two views, two stored procedures, two triggers and two functions. A graphical front end, user authentication, a transaction-history table and loan repayment schedules are outside the present scope. **Project planning involved six phases:** (1) requirement analysis; (2) ER design, keys and normalisation; (3) schema creation; (4) data insertion and CRUD; (5) advanced objects; and (6) testing, screenshots, report and GitHub publication.

1.2. Client Identification and Recognition of need: -

The primary clients are bank and branch managers, who need to know how many accounts, deposits and loans each branch holds. Secondary stakeholders are tellers and customer-service staff, loan officers, auditors and database administrators. Their needs are a single trustworthy record of customers and accounts, safe fund transfers, prevention of negative balances, a log of every balance change, and quick summaries per customer and per branch.

1.3. Recognition and Knowledge of relevant contemporary issues: -

Data integrity: a balance that is wrong by one transaction is a financial and legal problem. **Auditability:** regulators expect every balance change to be traceable. **Security and privacy:** names, addresses, e-mails and phone numbers are personal data; all data in this project is fictitious. **Digital banking growth:** online and mobile banking increase transaction volumes and the need for reliable back-end databases. **Rule enforcement:** rules such as overdraft control must hold at database level, not only in applications.

1.4. Task Identification: -

- **Design and DDL:** identify entities and keys, resolve M:N relations and the multivalued phone attribute; create the database, 10 tables, constraints and 3 indexes.
- **DML:** insert data; perform SELECT (WHERE, JOIN, GROUP BY, HAVING), UPDATE and DELETE operations.
- **Advanced objects:** create 2 views, 2 stored procedures, 2 triggers (with an audit table) and 2 functions.
- **Verification and publication:** verify every object, capture screenshots and publish the project on GitHub.

1.5. Technologies Used: -

Technology	Purpose in the project
MySQL (RDBMS)	Database engine that stores the tables and runs all SQL, views, routines and triggers.
SQL – DDL and DML	CREATE DATABASE / TABLE / INDEX / VIEW; INSERT, SELECT, UPDATE, DELETE with JOIN, GROUP BY and HAVING.
Stored routines, triggers	Procedures, functions (DELIMITER, DECLARE, IF…ELSE, OUT) and BEFORE / AFTER UPDATE triggers.
Git, GitHub	Version control and hosting of the SQL script, screenshots and README.

1.6. Timeline of the project

The project was carried out in the six phases of Figure 1. Earlier steps were revisited whenever a later step exposed a design problem.

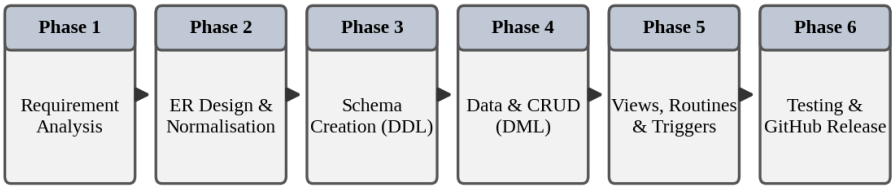


Figure 1: Development lifecycle of the project

CHAPTER 2.
LITERATURE REVIEW/BACKGROUND STUDY

2.1 Timeline of the reported problem:

Early banks kept paper ledgers; the first computerised systems of the 1960s used files and hierarchical or network databases tied to the physical layout of data. In 1970 E. F. Codd proposed the **relational model**, and SQL became its standard language in the 1980s. Chen's entity-relationship model (1976) let designers draw a database before building it, and the ACID properties (Haerder and Reuter, 1983) defined what "reliable" means for operations such as money transfer. From the 1990s, **core banking systems** have served all branches from one central database, today behind internet and mobile banking. The problems reported at every stage are similar: duplicated or inconsistent data, partial updates when a transfer fails halfway, balances changed without a record, and slow reporting.

2.2 Existing solutions:

Spreadsheets and flat files are easy to start with but have no constraints, no multi-user safety and no audit trail. **Commercial core banking platforms** (e.g. Oracle FLEXCUBE, Finacle, Temenos) are complete but licensed and opaque. **Application-only logic** (rules written only in application code) can be bypassed by direct database access. **A relational database with constraints, routines and triggers** keeps data, rules and audit in one place and is open and teachable – the approach chosen here.

2.3 Bibliometric analysis:

A review of the textbooks, papers and manuals in the References shows the following.

Key features

- Primary and foreign keys and UNIQUE, NOT NULL and DEFAULT constraints protect integrity at table level; junction tables resolve many-to-many relations, and separate tables hold multivalued attributes (third normal form).
- Stored procedures, functions and triggers move business rules into the database so that every application obeys them.

Effectiveness

- Constraints and triggers keep invalid data, such as a negative balance, out of the tables; views give managers simple summaries.

Drawbacks

- Logic inside the database is harder to version and test than application code, and careless joins can double-count values (see Section 4.1). A script-only project has no user interface, and sample data cannot show performance at real scale.

2.4 Review Summary:

The literature agrees that a sound banking database starts with a clear ER design, uses keys and constraints, is normalised, protects money movements with transactions and records changes in an audit trail. This project adopts these practices and lists explicit transactions and role-based security as future work.

2.5 Problem Definition:

To design and implement a relational database for a banking environment that **stores customers, accounts, branches and loans without redundancy**, supports **CRUD operations and reporting queries**, and uses **views, stored procedures, triggers and functions** to enforce business rules, prevent overdrafts, transfer money safely and keep an audit trail.

2.6 Goals/ Objectives:

- To design a normalised schema of 10 tables with correct keys and constraints.
- To demonstrate Create, Read, Update and Delete operations with joins, aggregation and filtering.
- To create views for customer-account and branch-performance reporting.
- To implement safe fund transfer and customer registration procedures.
- To enforce no-overdraft and audit-logging rules with triggers, and to calculate interest and total balances with functions.
- To publish the script, output screenshots and documentation on GitHub.

CHAPTER 3.
DESIGN FLOW/PROCESS

The design follows the standard database-development pipeline: understand requirements, draw the conceptual (ER) model, convert it to tables, implement the schema, populate and query it, add programmable objects and test.

3.1 Evaluation & Selection of Features: -

Feature	Reason	Priority	Decision
Core, lookup and junction tables with keys	Integrity; joint accounts and loans	High	Included
CRUD with JOIN, GROUP BY, HAVING	Daily operations and reports	High	Included
Views, procedures, functions, triggers	Reporting, safe transfer, overdraft rule, audit	High	Included
Transaction history, loan repayments	Statements and EMI tracking	Medium	Future
Roles, encryption, front-end UI	Security and usability	Medium	Future

3.2 Design Constraints: -

- **Data:** all data is fictitious; loan dates use CURDATE(), so they depend on the day the script is run.
- **Technical and integrity:** a standard MySQL server; routines and triggers need a DELIMITER change; foreign keys force tables to be created and filled parent-first.
- **Security and resources:** real data would need encryption and access control; this is a single-developer, script-based project with no front end.

3.3 Analysis of Features and finalization subject to constraints:

Every feature that can be built with plain MySQL was finalised. A transaction-history table, repayment schedules, role-based security and a front end were deferred. Business rules were placed in triggers and procedures so that applications cannot bypass them.

3.4 Design Flow:

Figure 2 shows the flow: requirements become an ER design; the design becomes a schema through DDL; data and CRUD are added; advanced objects implement the business logic; and the script, screenshots and report are delivered on GitHub. The conceptual model is shown in Figure 3.

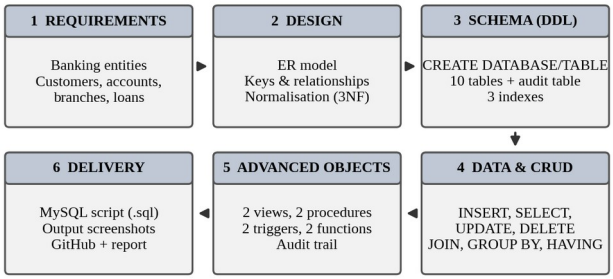


Figure 2: Workflow and architecture of the database project

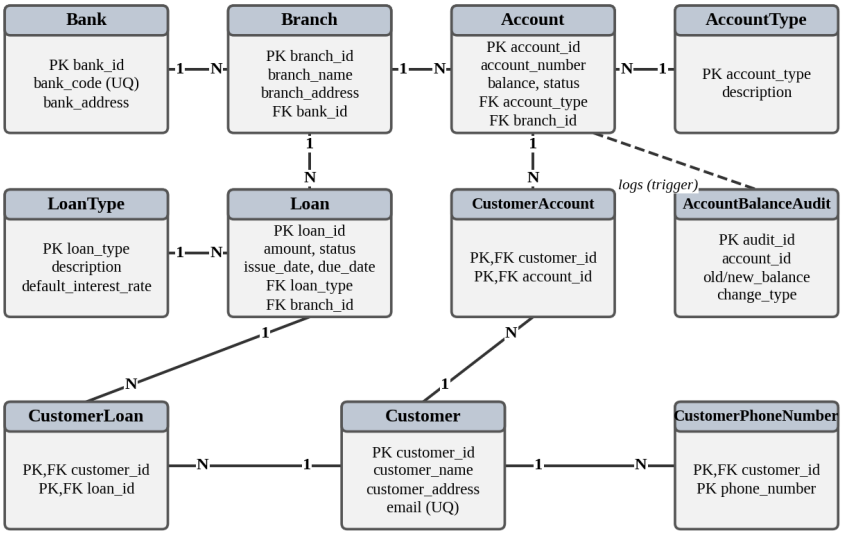


Figure 3: Entity-relationship (ER) diagram of the FinCore database

3.5 Design Selection: -

A normalised relational design was selected. A **bank has many branches**; a **branch has many accounts and loans**; and **customers can hold many accounts and loans while an account or loan may be shared by several customers** – an M:N relation stored in the junction tables CustomerAccount and CustomerLoan. Several phone numbers per customer are stored in CustomerPhoneNumber (composite key), types are lookup tables, and DECIMAL(12,2) is used for money to avoid rounding errors. Table 3.1 summarises the schema.

Table 3.1: Structure of the bank_management_db database

Table	Primary key	Foreign keys	Purpose
Bank	bank_id	–	Banks (bank_code UNIQUE)
Branch	branch_id	bank_id → Bank	Branches of a bank
Customer	customer_id	–	Customers; e-mail UNIQUE
CustomerPhoneNumber	customer_id, phone_number	→ Customer	Multivalued phone numbers
AccountType	account_type	–	Savings, Checking, Fixed Deposit
Account	account_id	account_type, branch_id	Accounts and balances
CustomerAccount	customer_id, account_id	→ Customer, → Account	M:N junction
LoanType	loan_type	–	Lookup with default rate
Loan	loan_id	loan_type, branch_id	Loans, dates, status, rate
CustomerLoan	customer_id, loan_id	→ Customer, → Loan	M:N junction
AccountBalanceAudit	audit_id (auto)	–	Audit log filled by trigger

3.6 Implementation plan/methodology

Development was incremental: each table was created and checked with DESCRIBE, data was inserted and verified with SELECT, and each advanced object was tested immediately. Table 3.2 lists the modules in the order implemented.

Table 3.2: Modules of the database script

No.	Module	Objects created	Output
1	Database, tables, indexes (DDL)	bank_management_db; 10 tables; 3 indexes	Figure 4
2	Insert and read data	10 INSERT; SELECT with WHERE, JOIN, GROUP BY, HAVING	Figures 5–7
3	Update and delete	4 UPDATE and 2 DELETE statements	Figure 8
4	Views	vw_CustomerAccountSummary, vw_BranchPerformance	Figure 9
5	Stored procedures	sp_TransferFunds, sp_AddCustomer	Figure 10
6	Triggers and audit table	tr_LogAccountBalanceChange, tr_PreventOverdraft	Figure 11
7	Functions	fn_CalculateMonthlyInterest, fn_GetCustomerTotalBalance	Figure 12

CHAPTER 4.
RESULTS ANALYSIS AND VALIDATION

4.1 Implementation of the solution:

The solution is implemented in the script *FinCore_Banking_System.sql* and run on MySQL. The output of each module is shown below with its interpretation. Money values are in the single currency of the sample data.

Schema creation and data. The database *bank_management_db* was created and the 10 tables were built parent-first; SHOW TABLES lists them and DESCRIBE confirms the structure (Figure 4). In *Account*, account_id is the primary key, account_number is UNIQUE, the foreign keys are indexed (MUL), and balance and status default to 0.00 and "Active". Ten INSERT statements then loaded the sample data summarised in Table 4.1.

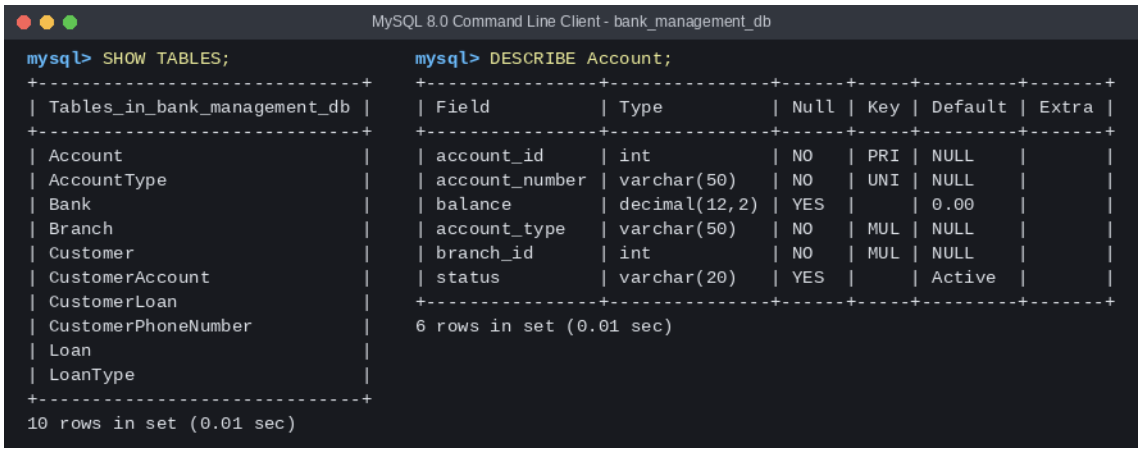


Figure 4: Tables created in bank_management_db (SHOW TABLES, DESCRIBE Account)

Table 4.1: Overview of the database and sample data

Item	Value
Banks / branches / customers (loaded) / phone numbers	2 (MAYBANK, CIMB) / 3 / 4 (+1 by procedure) / 5
Accounts (Savings, Checking, Fixed Deposit)	5 (2, 2, 1); total balance 56,700.00 initially
Loans (Personal, Home, Auto)	3 (1, 1, 1); total 585,000.00; terms 5, 20 and 5 years

Figure 5 shows the Account and Loan tables after insertion; the loan dates are generated with CURDATE().

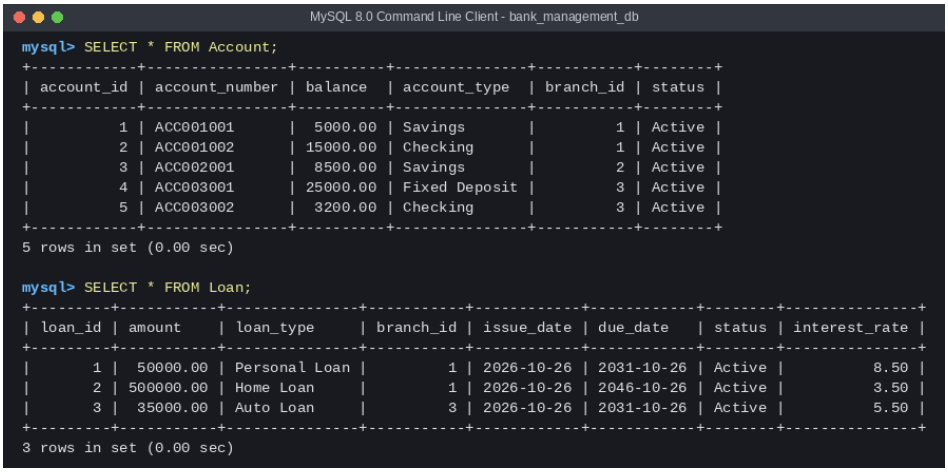


Figure 5: Sample data inserted into Account and Loan

Reading data (SELECT). The AND condition returns only ACC002001, because ACC001001 holds exactly 5,000. The three-table JOIN lists each customer's accounts with the branch name; Azhar bin Mohamed appears twice as he owns two accounts (Figure 6). The loan JOIN shows that Azhar holds the Personal and Home loans and Raj Kumar the Auto loan. The HAVING query keeps account types with an average balance above 5,000: Savings (6,750), Checking (9,100) and Fixed Deposit (25,000) (Figure 7). A LEFT JOIN count also showed two accounts at Maybank Pavilion and CIMB Mid Valley and one at Maybank KLCC.

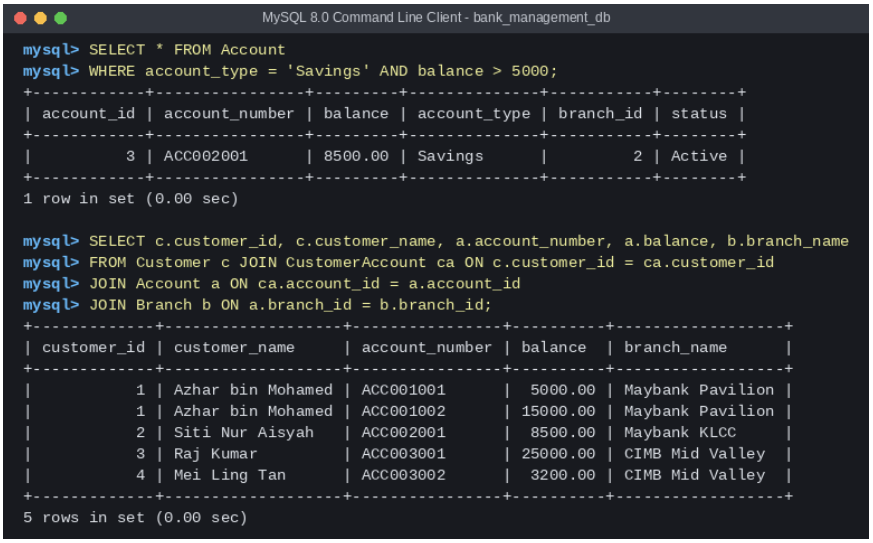


Figure 6: SELECT with AND condition and multi-table JOIN

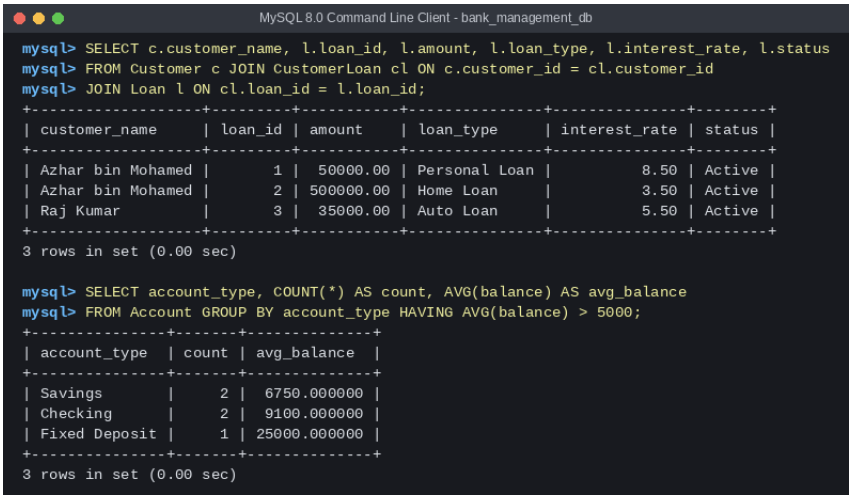


Figure 7: Loan JOIN and GROUP BY with HAVING

Update and delete. A deposit of 1,000 raised account 1 from 5,000 to 6,000; a multi-column UPDATE changed the e-mail and address of customer 1; and the test customer 99 was deleted, leaving an empty set (Figure 8). Every UPDATE and DELETE used a WHERE clause, so only the intended row was affected.

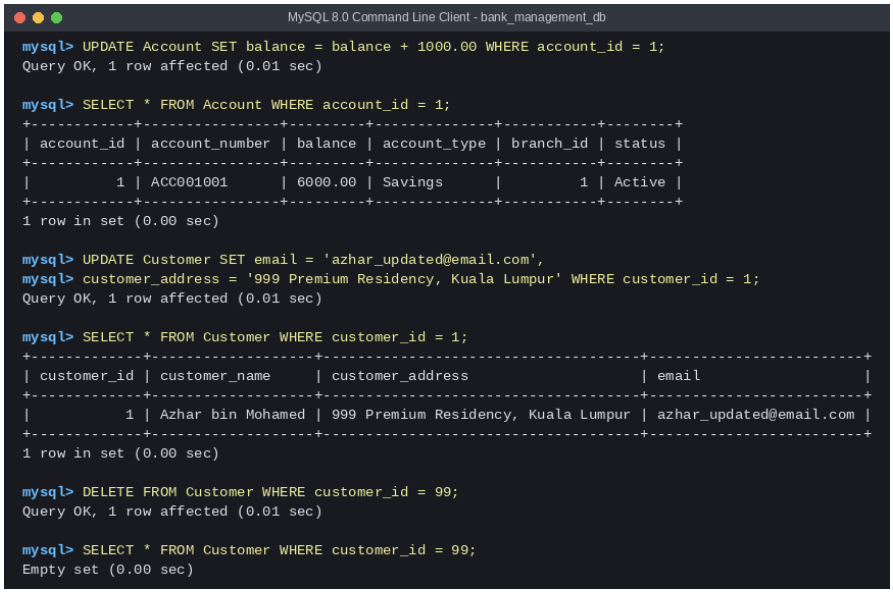


Figure 8: UPDATE and DELETE operations with verification

Views. *vw_CustomerAccountSummary* gives each customer's active accounts and total balance (customer 5, who has no account yet, shows 0 and NULL). *vw_BranchPerformance* summarises accounts, loans, deposits and loans issued per branch (Figure 9). *Observation:* because a branch's accounts and loans are joined together, each account row is repeated once per loan, so Maybank Pavilion shows deposits of 42,000.00 and loans of 1,100,000.00 although its

actual figures are 21,000.00 and 550,000.00 (CIMB Mid Valley's loans show 70,000.00 instead of 35,000.00). The DISTINCT counts are correct; the sums need separate sub-queries (Section 5.2).

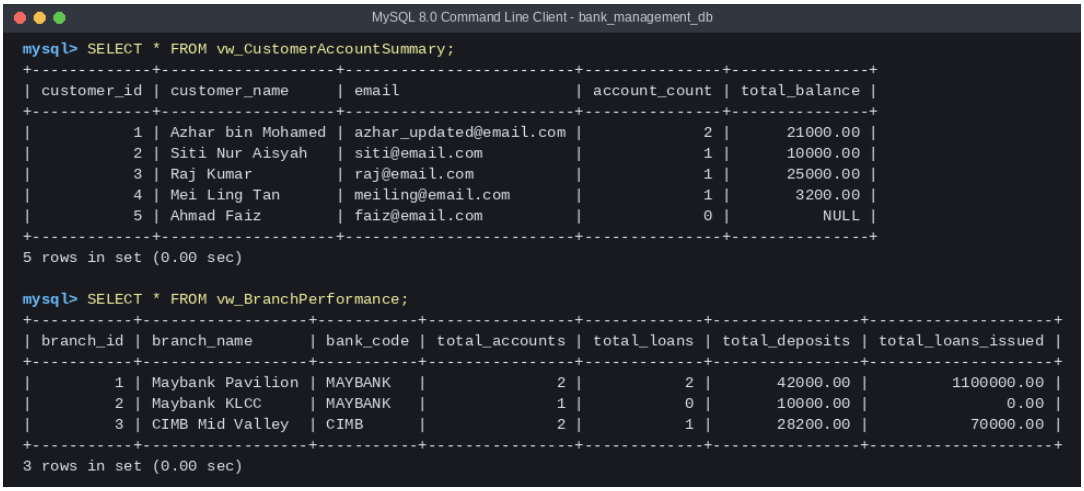


Figure 9: Output of the two views

Stored procedures. *sp_TransferFunds* moved 500.00 from account 1 to account 2 (6,000.00 / 15,000.00 became 5,500.00 / 15,500.00) and returned "SUCCESS: Transfer completed"; if the source balance were too low it would return "ERROR: Insufficient funds" and change nothing. *sp_AddCustomer* inserted customer 5, Ahmad Faiz (Figure 10).

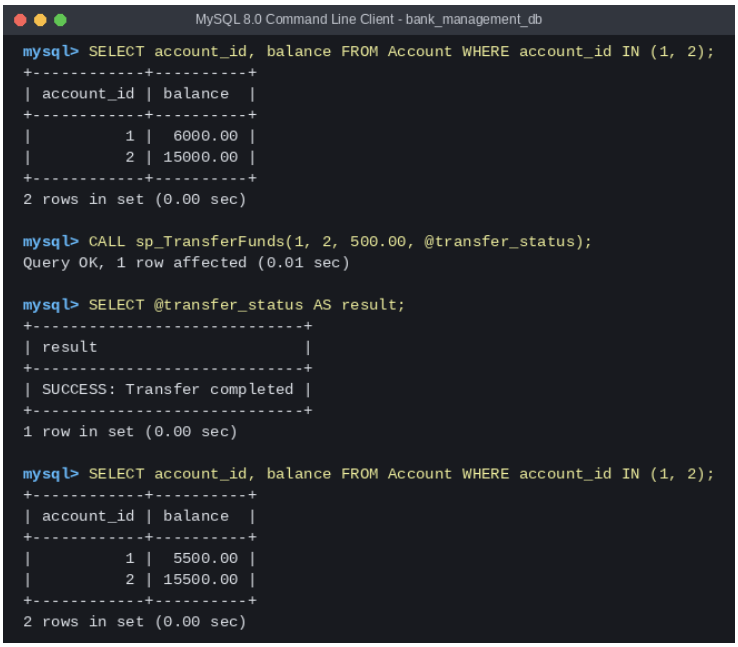


Figure 10: Stored procedure sp_TransferFunds

Triggers. *tr_LogAccountBalanceChange* fires after every update that changes a balance: a deposit of 2,000 and a withdrawal of 500 on account 3 produced two audit rows with old and new balances, change type and time. *tr_PreventOverdraft* fires before the update and raises SQLSTATE 45000 if the new balance would be negative, so a withdrawal of 999,999.00 is rejected (Figure 11). The transfer in Figure 10 ran before the audit trigger was created, so it was not logged.

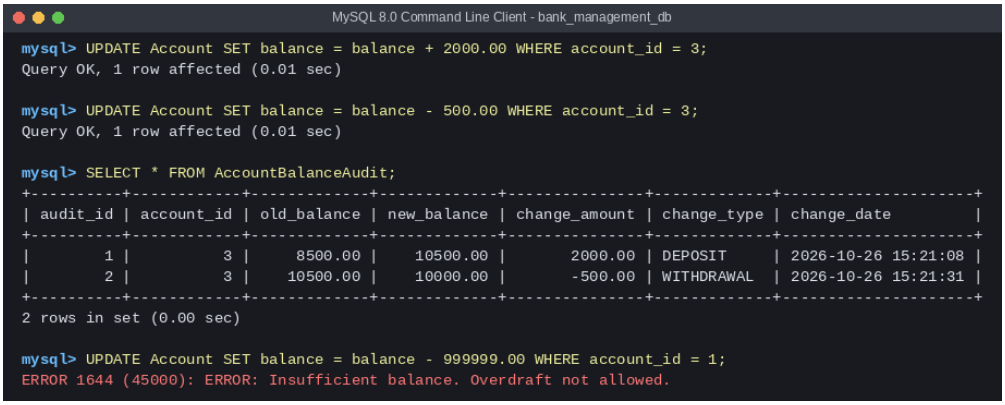


Figure 11: Trigger audit log and overdraft protection

Functions. *fn_CalculateMonthlyInterest* returns balance × rate ÷ 100 ÷ 12; at 2.5%, a balance of 5,500.00 earns 11.46 a month and 10,000.00 earns 20.83. *fn_GetCustomerTotalBalance* adds a customer's active-account balances; the totals (21,000.00, 10,000.00, 25,000.00, 3,200.00 and 0.00) add up to the bank's 59,200.00 (Figure 12).

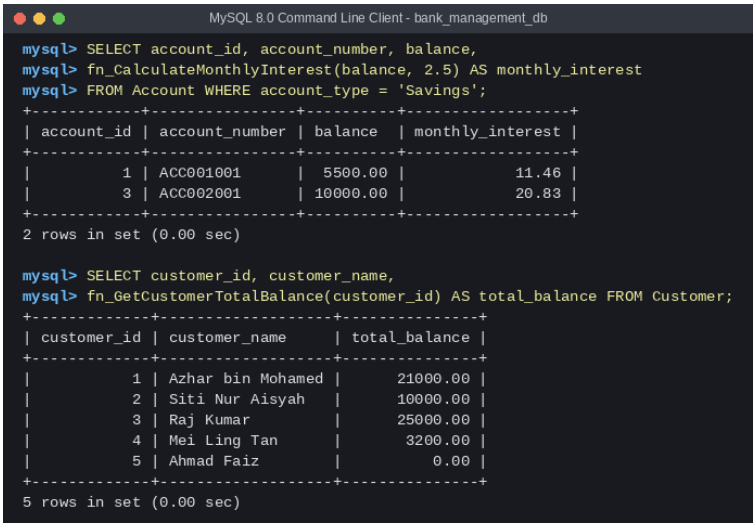


Figure 12: Output of the two user-defined functions

4.2 Summary of findings and validation:

Key findings

- The schema of **10 tables + 1 audit table** is normalised; M:N relations and multivalued data are in separate tables and lookup tables control allowed values.
- All CRUD operations work, and each is verified by a follow-up SELECT.
- Money movement is protected: the transfer procedure checks the balance and the **overdraft trigger** blocks negative balances; the **audit trigger** records every balance change.
- Total deposits reconcile: 56,700.00 + 1,000 + 2,000 – 500 = **59,200.00**, equal to the sum of the function outputs (a transfer leaves the total unchanged).
- One weakness was found: the branch-performance view double-counts sums because of join fan-out.

Validation

Table 4.2: Test summary

No.	Test case	Expected result	Result
1	Create database and 10 tables	SHOW TABLES lists 10 tables	Pass
2	Insert parent rows before child rows	No foreign-key error	Pass
3	SELECT with AND, JOIN, GROUP BY, HAVING	Correct rows and counts (Figs. 6–7)	Pass
4	UPDATE / DELETE with WHERE	Only the target row changes (Fig. 8)	Pass
5	Views	Summaries per customer and branch (Fig. 9)	Pass*
6	sp_TransferFunds 500.00	Debit and credit; total unchanged	Pass
7	Audit trigger (deposit, withdrawal)	Two audit rows with correct type	Pass
8	Overdraft of 999,999.00	Error 45000; balance unchanged	Pass
9	Functions	Interest 11.46 / 20.83; totals reconcile	Pass

**Pass, with the fan-out observation noted for vw_BranchPerformance.*

CHAPTER 5.
CONCLUSION AND FUTURE WORK

5.1 Conclusion: -

This project achieves its objectives: a normalised MySQL database for a banking environment was designed, implemented, populated, queried and verified. It stores banks, branches, customers, accounts and loans with correct keys and constraints; supports all CRUD operations; and uses views, procedures, triggers and functions for reporting, safe fund transfer, overdraft prevention, an automatic audit trail and interest and balance calculations. The project also gave practical experience of the complete database life cycle, from requirements and ER design to testing and publication on GitHub.

Some limitations remain. The transfer procedure does not use an explicit transaction and does not check that the destination account exists or that the amount is positive; there is no transaction-history table, so balance changes are visible only through the audit table; *vw_BranchPerformance* double-counts when a branch has several accounts and loans; loan dates depend on the day of execution; and there is no user security or front end. The data is fictitious, so the work illustrates the method rather than a real bank.

5.2 Future Scope: -

- Wrap the transfer in START TRANSACTION / COMMIT / ROLLBACK and validate the destination account and a positive amount.
- Correct *vw_BranchPerformance* by aggregating accounts and loans in separate sub-queries.
- Add a Transaction table, loan repayment (EMI) tables, role-based security (GRANT/REVOKE) and encryption of personal data.
- Build a web or mobile front end connected to the database, and schedule monthly interest posting and backups.

APPLICATION CODE OF PROJECT

The main parts of the SQL script are given below. The complete script, *FinCore_Banking_System.sql* (10 tables, 10 INSERT statements, CRUD examples, the second procedure and function, and verification queries), is available in the GitHub repository.

```
-- 1. Database and key table (DDL)
CREATE DATABASE IF NOT EXISTS bank_management_db;  USE bank_management_db;
CREATE TABLE Account (
    account_id INT PRIMARY KEY,
    account_number VARCHAR(50) NOT NULL UNIQUE,
    balance DECIMAL(12,2) DEFAULT 0.00,
    account_type VARCHAR(50) NOT NULL,  branch_id INT NOT NULL,
    status VARCHAR(20) DEFAULT 'Active',
    FOREIGN KEY (account_type) REFERENCES AccountType(account_type),
    FOREIGN KEY (branch_id) REFERENCES Branch(branch_id));
-- (Bank, Branch, Customer, Loan, junction tables, etc. follow the same pattern)

-- 2. View: customer account summary
CREATE VIEW vw_CustomerAccountSummary AS
SELECT c.customer_id, c.customer_name, c.email,
       COUNT(ca.account_id) AS account_count, SUM(a.balance) AS total_balance
FROM Customer c LEFT JOIN CustomerAccount ca ON c.customer_id = ca.customer_id
LEFT JOIN Account a ON ca.account_id = a.account_id
WHERE a.status = 'Active' OR a.status IS NULL
GROUP BY c.customer_id, c.customer_name, c.email;

-- 3. Stored procedure: transfer funds
DELIMITER //
CREATE PROCEDURE sp_TransferFunds(IN p_from_account_id INT, IN p_to_account_id INT,
                                  IN p_amount DECIMAL(12,2), OUT p_status VARCHAR(100))
BEGIN
  DECLARE v_from_balance DECIMAL(12,2);
  SELECT balance INTO v_from_balance FROM Account WHERE account_id = p_from_account_id;
  IF v_from_balance < p_amount THEN SET p_status = 'ERROR: Insufficient funds';
  ELSE
    UPDATE Account SET balance = balance - p_amount WHERE account_id = p_from_account_id;
    UPDATE Account SET balance = balance + p_amount WHERE account_id = p_to_account_id;
    SET p_status = 'SUCCESS: Transfer completed';
  END IF;
END//
DELIMITER ;

-- 4. Triggers: audit log and overdraft prevention
DELIMITER //
CREATE TRIGGER tr_LogAccountBalanceChange AFTER UPDATE ON Account FOR EACH ROW
BEGIN
  IF OLD.balance <> NEW.balance THEN
    INSERT INTO AccountBalanceAudit (account_id, old_balance, new_balance, change_amount, change_type)
    VALUES (NEW.account_id, OLD.balance, NEW.balance, NEW.balance - OLD.balance,
            IF(NEW.balance > OLD.balance, 'DEPOSIT', 'WITHDRAWAL'));
  END IF;
END//
CREATE TRIGGER tr_PreventOverdraft BEFORE UPDATE ON Account FOR EACH ROW
BEGIN
  IF NEW.balance < 0 THEN
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'ERROR: Insufficient balance. Overdraft not allowed.';
  END IF;
END//
DELIMITER ;

-- 5. Function: monthly interest
DELIMITER //
CREATE FUNCTION fn_CalculateMonthlyInterest(p_balance DECIMAL(12,2), p_annual_rate DECIMAL(5,2))
RETURNS DECIMAL(12,2) DETERMINISTIC
BEGIN
  RETURN (p_balance * p_annual_rate / 100) / 12;
END//
DELIMITER ;
```

REFERENCES

1. Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377–387.
2. Chen, P. P. (1976). The entity-relationship model – toward a unified view of data. ACM Transactions on Database Systems, 1(1), 9–36.
3. Haerder, T., & Reuter, A. (1983). Principles of transaction-oriented database recovery. ACM Computing Surveys, 15(4), 287–317.
4. Elmasri, R., & Navathe, S. B. (2016). Fundamentals of Database Systems (7th ed.). Pearson.
5. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). Database System Concepts (7th ed.). McGraw-Hill.
6. Beaulieu, A. (2020). Learning SQL (3rd ed.). O'Reilly Media.
7. Oracle Corporation. MySQL 8.0 Reference Manual. https://dev.mysql.com/doc/refman/8.0/en/

APPENDIX

Design Checklist

- **Requirements and design:** entities and operations identified; ER diagram drawn; M:N relations and the multivalued phone attribute resolved; keys chosen.
- **Schema and data:** 10 tables, constraints, defaults and 3 indexes created parent-first; sample data inserted; CRUD verified.
- **Advanced objects and publication:** 2 views, 2 procedures, 2 triggers and 2 functions tested; script, screenshots and README uploaded to GitHub.

USER MANUAL

A. Setting up the project

1. Install MySQL Server and a client (MySQL Workbench or the mysql command-line client).
2. Download or clone the GitHub repository and open FinCore_Banking_System.sql.
3. Run the script from top to bottom; it creates *bank_management_db*, the tables, sample data and all advanced objects.

B. Using the database

1. Run SHOW TABLES and DESCRIBE to inspect the structure, and SELECT * FROM each table to see the data.
2. Query *vw_CustomerAccountSummary* and *vw_BranchPerformance* for reports.
3. Transfer money with CALL sp_TransferFunds(from_id, to_id, amount, @status); SELECT @status;
4. Add a customer with CALL sp_AddCustomer(id, name, address, email); check SELECT * FROM AccountBalanceAudit; after any balance change.

GITHUB REPOSITORY

The complete project (SQL script, output screenshots and README) is published at: https://github.com/jindalbhumi125-dev/FinCore-Banking-System

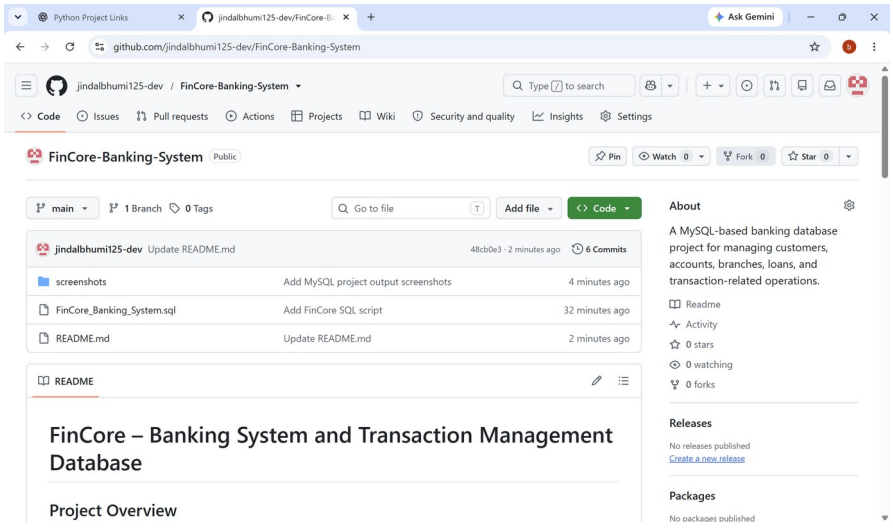


Figure 13: GitHub repository of the project